

1. Présentation

Un parcours de graphe est un algorithme consistant à explorer (on dit aussi visiter) tous les sommets d'un graphe de proche en proche à partir d'un sommet initial. Les algorithmes de parcours d'un graphe sont à la base de nombreux algorithmes très utilisés : routage des paquets de données dans un réseau, découverte du chemin le plus court pour aller d'une ville à une autre...

Il existe 2 méthodes pour parcourir un graphe :

- Le parcours en largeur d'abord
- le parcours en profondeur d'abord

Tous les parcours suivent plus ou moins le même algorithme de base :

On visite un sommet s_1 . On note S l'ensemble des voisins de s_1 .

Tant que S n'est pas vide :

- *on choisit un sommet s de S ;*
- *on visite s ;*
- *on ajoute à S tous les voisins de s pas encore visités.*

2. Marquage/mémorisation des sommets visités/découverts

La dernière étape implique que l'on retienne les sommets déjà **visités** ou **découverts**.

Vocabulaire :

- **Lorsqu'on visite un sommet**, nous dirons que le sommet est « **visité** » ;
- **Lorsqu'on ajoute un sommet à S** : nous dirons que le sommet est « **découvert** ».

Pour retenir les sommets déjà **visités** ou **découverts**, on utilise un **marquage** qui peut être :

- une coloration du sommet (par exemple **blanc** : sommet non visité/découvert, **noir** : sommet déjà visité/découvert) ;
- une liste ;
- ou, plus efficace, un dictionnaire des sommets déjà visités/découverts : pour un parcours simple, on ne s'intéresse alors qu'aux clés du dictionnaire et les valeurs associées sont sans importance (None, True ...), mais on peut aussi stocker comme valeur le précédent sommet dans le parcours, ce qui permettra de reconstituer un chemin.

3. Parcours en largeur et en profondeur

La différence de parcours se fait sur le choix du sommet s et sur la structure de l'ensemble S .

Parcours en largeur

Si on utilise une **file** pour S , les sommets enregistrés dans S en premier vont être visités les premiers.

On va donc visiter d'abord les sommets le plus prêts de s_1 (ceux à distance 1, puis ceux à distance 2, puis 3...).

C'est un **parcours en largeur (BFS, Breadth First Search)**.

On va utiliser ce parcours pour trouver le sommet le plus prêt de s_1 vérifiant une condition donnée ou le chemin le plus court entre s_1 et un autre sommet.

On marque les sommets découverts afin que chaque sommet ne soit enfilé qu'une fois.
Ainsi, lorsqu'on défile un sommet, on sait qu'il n'a pas encore été visité.

```
parcours_largeur(G, s):
```

```
  f = file_vide()
```

```
  enfiler(f, s)
```

```
  découverts = {s}
```

```
  tant que f n'est pas vide :
```

```
    s = défiler(f)
```

```
    afficher s
```

```
    pour v dans voisins(s):
```

```
      si v n'est pas dans découverts :
```

```
        ajouter v à découverts
```

```
        enfiler(f, v)
```

ou marquer s comme découvert

ou marquer v comme découvert

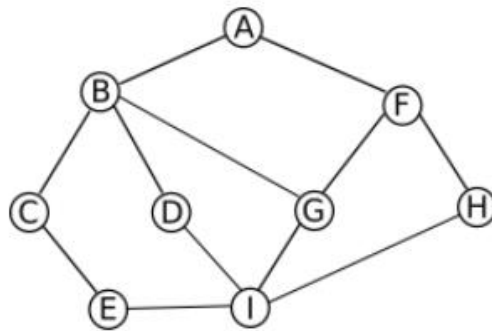
Exercice 1

Appliquez l'algorithme du parcours en largeur d'abord au graphe ci-dessous.

Le "point de départ" de notre parcours sera le sommet A.

Vous noterez les sommets atteints à chaque étape ainsi que les sommets présents dans la file f.

Vous pourrez aussi, à chaque étape, donner les changements de couleur des sommets.



Solution : (ce n'est pas la seule solution possible)

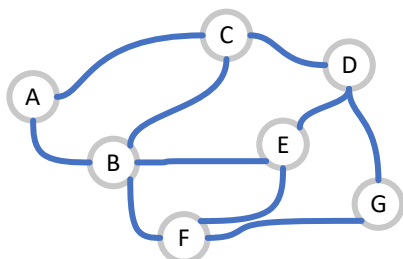
A, B, F, C, D, G, H, E et I

Dans le cas d'un **parcours en largeur**, on "découvre" d'abord tous les sommets situés à une distance k du sommet "origine" avant de commencer la découverte des sommets situés à une distance $k + 1$ (on définit la distance comme étant le nombre d'arêtes à parcourir depuis A pour arriver à destination).

En effet, dans l'exercice précédent, nous avons bien :

sommets	A	B	F	C	D	G	H	E	I
distances depuis A	0	1	1	2	2	2	2	3	3

Exercice 2



Donner un exemple de parcours en largeur du graphe suivant en partant :

a. du sommet A ;

b. du sommet B.

Solutions : a. A-B-C-F-E-D-G

b. B-A-C-E-F-D-G ou encore B-A-F-E-C-G-D

Parcours en profondeur

Si on utilise une **pile** pour S, les sommets enregistrés en dernier vont être visités en premier : on parcourt le graphe en visitant à chaque fois un voisin du dernier sommet, sauf si celui-ci n'a pas de voisin non visité, auquel cas on remonte au dernier sommet ayant un voisin non visité.

C'est un **parcours en profondeur (DFS, Depth First Search)**.

C'est le parcours utilisé naturellement par une personne qui **explore un labyrinthe**.

Dans un parcours en profondeur, **on marque les sommets visités**.

Un même sommet peut être empilé plusieurs fois.

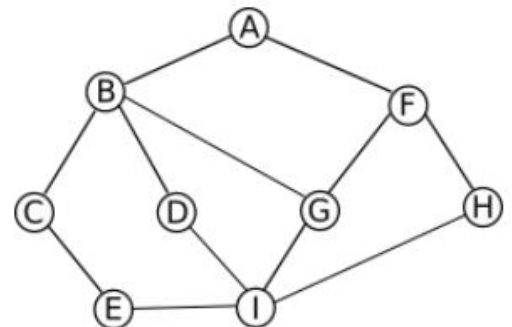
En effet, un parcours en profondeur doit permettre de passer d'un sommet à un sommet voisin, et si ce sommet a déjà été empilé en début de parcours, il doit être empilé de nouveau pour devenir immédiatement accessible.

```
parcours_profondeur(G, s):  
  p = pile_vide()  
  empiler(p, s)  
  visités = {}  
  Tant que p n'est pas vide :  
    s = depiler(p)  
    si s n'est pas déjà visité :  
      afficher s  
      ajouter s à visités  
      pour v dans voisins(s):  
        si v n'est pas dans visités :  
          empiler(p, v)
```

Vous avez sans doute remarqué que l'algorithme du parcours en profondeur ressemble beaucoup à l'algorithme du parcours en largeur, on a juste remplacé la file par une pile.

Exercice 1

Appliquez l'algorithme du parcours en profondeur au graphe ci-contre.



Solution : (ce n'est pas la seule solution possible)

A, B, C, E, I, D, G, F et H

Exercice 2

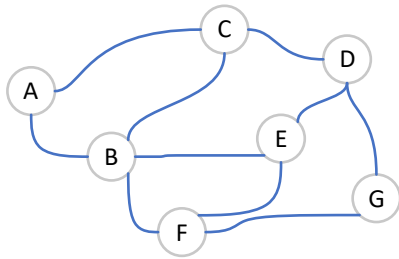
Comparez le résultat obtenu dans le graphe de l'exercice précédent avec le parcours en largeur (A, B, F, C, D, G, H, E et I) et le résultat obtenu avec le parcours en profondeur (A, B, C, E, I, D, G, F et H).

Solution :

Dans le cas du parcours en largeur on "découvrirait" tous les sommets situés à une distance k de l'origine avant de s'intéresser aux sommets situés à une distance $k + 1$ de l'origine.

Dans le cas du parcours en profondeur, on va chercher à aller "le plus loin possible" dans le graphe : $A \rightarrow B \rightarrow C \rightarrow E \rightarrow I \rightarrow D$. Quand on tombe sur "un cul-de-sac" (dans notre exemple, D est un "cul-de-sac", car une fois en D, on peut uniquement aller en B, or, B a déjà été visité...), on revient "en arrière" (dans notre exemple, on repart de B pour aller explorer une autre branche : $G \rightarrow F \rightarrow H$)

Exercice 3



Donner un exemple de parcours en profondeur du graphe suivant en partant :

- du sommet A ;
- du sommet F.

Solution :

a. A-B-C-D-E-F-G

b. F-B-C-D-G-E-A

4. Parcours récursif

```
parcours(G, s, visités):  
    ajouter s à visités  
    afficher s  
    pour v dans voisins(s):  
        si v n'est pas dans visités :  
            parcours(G, v, visités)
```

C'est un parcours en

La condition d'arrêt est implicite : si tous les sommets sont déjà visités, les appels s'interrompent.

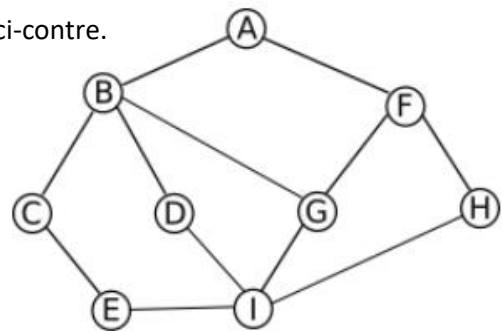
Ce parcours récursif ne s'effectue pas dans le même ordre que la version itérative.

En effet, le premier voisin va être visité d'abord, alors que la version itérative visite le dernier voisin d'abord.

Vous pouvez remarquer l'extrême simplicité de cet algorithme (au niveau de sa conception) : c'est souvent le cas avec les algorithmes récursifs.

Exercice :

Appliquez l'algorithme récursif du parcours en profondeur au graphe ci-contre.



Solution : (ce n'est pas la seule solution possible)

A, B, C, E, I, D, G, F et H